booking — arhitectura țintă (ce trebuie să fie la final, după F15 și U5)

Doar Docker: docker compose cu profiluri pe EC2, fără Kubernetes · F = feature-ul de backend, U = pasul de interfață din PLAN.pdf · eticheta = profilul compose · chenar punctat = doar local / pe instanța de demo

Browser (Internet)

https://<domeniu> (F5)
certificat certbot
/ · /api · /mcp · /auth · /grafana

EC2 · security group
80, 443 publice (F5)
22 doar de la IP-ul lui Rareș (F5)

Clienți: booking-client în browser (U1–U5)
Postman (F1) · MCP Inspector (F7)
token din /api/auth/login (F3) → de la Keycloak (F14)

Claude API (F10) · ANTHROPIC_API_KEY doar în .env
chemat de booking-agent prin Spring AI ChatClient
fără cheie / fără răspuns: fallback pe reguli

Spring AI (F10)

GitHub · monorepo-ul booking
Actions pe folder (F5): api (F5) · agent (F9)
notifications (F12) · ui (U3)
mvn -B verify (Testcontainers) → imagine

GHCR · imaginile :latest + :SHA (F5)

EC2 își trage singur imaginile (săgeata albastră): cron la 2 minute, deploy.sh (F5)

Docker Engine · docker compose (booking-ops/compose.yaml) · pe EC2: core · obs · agent · auth · edge · local și pe instanța de demo: toate profilurile

edge · rețeaua Docker *edge* (F15) · singurele porturi publicate: 80 / 443 (nginx)

nginx + certbot (F5) core · TLS
/ → ui · /api, /mcp → Kong (F15)
/auth → Keycloak · /grafana → oauth2-proxy (F14)

ui · booking-client (U1–U5) core
React + Vite + TS · nginx static (U3)

Kong — API gateway (F15) edge
DB-less: booking-ops/kong/kong.yml
/api, /mcp → booking-api · /api/notifications → notifications
jwt (cheia realm-ului) · rate-limiting (60/min availability)
correlation-id · prometheus · rețele edge + backend

oauth2-proxy (F14) auth
/grafana → Grafana
Kibana · Kafka UI
login Keycloak, doar ADMIN (altfel 403)
rețele edge + backend

/auth

/grafana

OIDC

auth (F14) · Keycloak = sursa conturilor și a rolurilor

Keycloak auth · --import-realm
KC_HTTP_RELATIVE_PATH=/auth
rețele edge + backend, fără ports

Postgres-ul Keycloak auth

realm-booking.json în booking-ops/keycloak/ · roluri CUSTOMER / BARBER / ADMIN / AGENT · clienți: booking-api · booking-agent (client credentials) · oauth2-proxy · booking-ui (PKCE, U5) · mapperi: permissions, phone_number

MCP prin Kong (F15)

aplicațiile tale · rețeaua Docker *backend* (F15) · fără ports · intră doar prin Kong

booking-agent · primul microserviciu (F9–F10) agent

Client MCP · cont AGENT din .env (F9)
→ client credentials de la Keycloak (F14) · prin Kong (F15)

RuleBasedPlanner (F9)
regula 1 (± 60 min) · regula 2 · @Scheduled la 5 min

LlmPlanner (F10)
ChatClient → Claude · ieșire structurată în record
timeout 20 s → fallback pe RuleBasedPlanner

fără bază de date · nu cunoaște REST-ul, doar tool-urile MCP
booking.agent.planner = RULES | LLM
local / demo: @KafkaListener pe NEEDS_RESCHEDULE și CANCELED (F11), nu timer
runId în MDC · reloghează la 401 / token aproape expirat

booking-api · monolitul (F1–F15) core

REST /api (F1, F3, F4, F6, F8)

Server MCP /mcp · Spring AI (F7–F8)

filtru JWT (F3) → **resource server pe issuer-ul Keycloak** (F14) · permisiunile în service: access.require(actor, …)

frizeri, proceduri (F1) · program, time-off (F1, urgent F8) · programări (F1) · conturi (F3, F14) · recurente + job (F4)

ocupare (F6) · **propuneri** (F8) planner (F10) · BookingRules (F1) · Actor, SYSTEM (F3)

AFTER_COMMIT → KafkaTemplate (F11) · booking.events.enabled = false pe EC2

SLF4J + MDC runId / seriesId, JSON ecs (F4, pe EC2 de la F5) · correlation-id de la Kong în MDC (F15)
Actuator + Micrometer pe 8081, nepublicat (F6) · OpenTelemetry OTLP → Tempo (F13, oprit pe EC2)

accept pe o propunere cheamă updateAppointment: aceeași cale și aceeași validare ca reprogramarea de mână; agentul doar propune, decide clientul (F8)

NEEDS_RESCHEDULE · CANCELED

messaging (F11–F12) · doar local / demo

Kafka apache/kafka · KRaft, un nod (F11) messaging

topic **booking.appointment-events** · cheie = appointmentId

topic **booking.proposal-events** · cheie = appointmentId

kafka-init creează topicurile (F11) messaging

contractul dintre servicii = topicurile · booking-docs/events.md, versionat (F12)

booking-notifications (F12) messaging

NotificationListener · consumer group propriu, offset la repornire
UQ pe event_id = consumator idempotent (F11)
GET /api/notifications/me · recipient_email = email din token

se salvează și se loghează, nu se trimite · nu citește baza API-ului, API-ul nu-l cheamă

data · volume Docker numite

MySQL 8.4 · booking_api_db (Flyway F2, în compose F5) core · Flyway V1–V9 · ddl-auto: validate
healthcheck → depends_on: service_healthy (F5) · @Lock(PESSIMISTIC_WRITE) + READ COMMITTED pe rezervare (F5)

booking_notifications_db (F12) · V1__notifications în serviciu, V8 o scoate din API messaging

booking-agent n-are bază · Postgres-ul Keycloak e în profilul auth · parolele doar în .env pe EC2 și local, în Git doar .env.example

monitoring (F6 · F13)

Prometheus (F6) obs · Grafana (F6) obs · Tempo (F13) logs

scrape: api:8081 (management, nepublicat) · Kong (F15) · Grafana: datasource + dashboard din booking-ops/grafana/, Tempo ca sursă (F13) · ocupare, statusuri, anulări târzii, booking_mcp_calls_total, tokeni și latența modelului · până la F14 Grafana pe 127.0.0.1:3000 + tunel ssh

logging (ELK · F13) · doar local / demo

Elasticsearch logs · Kibana logs · Filebeat logs

Filebeat citește logurile JSON ale containerelor → Elasticsearch → Kibana · căutare după runId, appointmentId sau traceId · traceId pe fiecare linie, în toate cele trei aplicații · Elasticsearch cere 1–2 GB

Ce ține locul Kubernetes (fără cluster)

rețele Docker (F15): edge = nginx, Kong, oauth2-proxy, ui · backend = api, agent, notifications, MySQL, Kafka, Keycloak · în ambele: Kong, oauth2-proxy, Keycloak

profiluri compose
core · obs · agent · messaging · logs · auth · edge
alese prin COMPOSE_PROFILES în .env

healthcheck + depends_on service_healthy (F5)
aplicațiile fără ports (F15)

config în Git (booking-ops/): kong/kong.yml · keycloak/realm-booking.json
grafana/ · compose.yaml · deploy.sh · .env.example

Memoria: pe EC2-ul obișnuit rulează core + obs + agent, apoi auth + edge; Kafka, booking-notifications și ELK + Tempo (messaging, logs) nu încap: local sau pe instanța de demo pentru susținere · pe EC2 booking.events.enabled = false (agentul rămâne pe @Scheduled) și exportul OTLP e oprit

Fluxul de livrare (F5): 1. git push pe main → 2. Actions: mvn -B verify cu Testcontainers, un job per folder (api · agent · notifications · ui) → 3. imaginea în GHCR (:latest + :SHA) → 4. pe EC2, cron la 2 minute: deploy.sh = git pull + docker compose pull + up -d (doar COMPOSE_PROFILES din .env) · Actions nu folosește ssh

Legendă
portocaliu = edge / gateway (domeniu, EC2, nginx, Kong, oauth2-proxy) · roșu = Keycloak și logurile · mov = metrici și tracing · verde = Kafka · galben = date · albastru = Docker în locul Kubernetes · violet MCS = aplicațiile tale
Partea IV: **F14** Keycloak (realm în Git, API și notifications = resource server, agentul pe client credentials, login și register scoase, Keycloak publicat prin nginx pe /auth, Grafana în spatele oauth2-proxy pe /grafana) · **F15** Kong = API gateway (jwt, rate-limiting, correlation-id, prometheus; agentul și ui-ul trec și ele prin Kong) + rețelele Docker: aplicațiile fără ports
Partea V: ui = booking-client (React + Vite + TS) · U1 rezervare (după F1, local) · U2 cont + programările mele (după F3, local) · U3 pagini + ziua salonului (după F6, ui-ul pe EC2) · U4 accept ora propusă (după F8) · U5 login Keycloak PKCE (imediat după F14, înainte de F15)